Interaktivt webbprojekt

Säkerhet

Sårbarhetsrisk

1. XSS

2. Osäker lagring av lösenord

3. Bristande validering av användarinmatning

Om webbplatsen skulle ha användarkonton bör lösenord aldrig lagras i klartext. I stället bör de hash-as och saltas, exempelvis med algoritmer som bcrypt.

Inmatningsfältet `#inputMeddelande` – risk för DOM-baserad XSS. DOM-baserad XSS kan uppstå. Ingen server är inblandad, datan skulle kunna komma via ingripandens inmatning i input-fältet, JS i användarens webbläsare skulle kunna stoppa in en skadlig kod in i sidan via .innerHTML och webbläsaren tolka det som kod och manipulera DOM.

Problem fanns i koden:

```
document.getElementById("output").innerHTML = meddelande;
```

En DOM-baserad attack skickas aldrig till servern och är svåra att upptäcka av brandväggar för webbaserade program (WAF) och syns inte i serverloggar.[1]

Åtgärder

1. Först testade jag vanlig användar-/text-input i inmatningsfältet för att testa funktionen.

2. Därefter matade jag in:

a) `<script>alert('Du har blivit hackad!')</script>`

Chrome visar inte någon alert popp up men input via .innerHTML. Input tolkas här som HTML-kod och webbläsaren bygger DOM-element av det. Detta syns i Dev Tools:

```
<button onclick="visaMeddelande()">Visa</button>
▼<div id="output"> == $0
  <script>alert('Du har blivit hackad!')</script>
```

Skriptet körs inte automatiskt eftersom det finns säkerhetsskydd i moderna webbläsare.

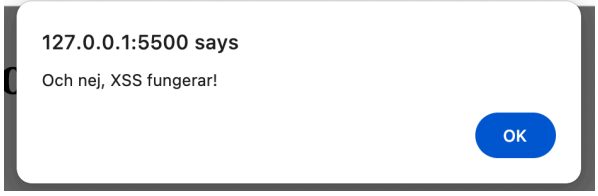
Däremot kan HTML-attribut, som innehåller JS, med händelser som operor, onclick och oljad fortfarande triggas.

Därför testade jag också: `<IMG SRC=/`

[1] Vad är en XSS-attack? Definition och förklaring. https://www.kaspersky.se/resource-center/definitions/what-is-a-cross-site-scripting-attack (hämtad 2026-03-14)

`onerror="alert(String.fromCharCode(88,83,83))"></img>`[2]

och `<img src=x onerror="alert('Och nej, XSS fungerar!')">`



<img onerror>- testet fungerar som XSS. När element skapas ser webbläsaren att det är ett script-element och innehållet skickas till JS-motorn och skriptet kan aktiveras automatisktSäkerhetsåtgärd 1

I stället för innerHTML lägger jag in .textContent i JS, för då tolkas inte input som HTML utan som ren text.

document.getElementById("output").*textContent* = meddelande;

Det skapas inget DOM-element som kan trigga onerror.

Säkerhetsåtgärd 2

Ett extra skyddslager är Content Security Policy (CSP) I <head>:

```
<meta http-equiv="Content-Security-Policy" content="default-
```

[2] OWASP. XSS Filter Evasion Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html (hämtad 2026-03-14)

`src 'self'; script-src 'self'; img-src 'self'; style-src 'self';">`

Denna CSP-regel kontrollerar vilka resurser såsom skript, stilar m.m. webbplatsen får ladda. Resurser som laddas får bara ha samma ursprung/domän, och resurser från externa servar är inte tillåtna som standard.
JS får bara köras från samma webbplats, extern JS blockeras. Bilder får bara laddas från samma webbplats, medan bilder från andra domäner blockeras. Stilar (CSS) får endast ha samma ursprung som webbsidan, och eftersom styles.css ligger i samma domän som resten av webbplatsen kommer det att godkännas av CSP:n.
Men inline CSS (t.ex. `<style> body {background-color: #f5f5f5; } </style>` och inline style-attribut: (t.ex. `<div style="background-color: #f5f5f5;">`

Detta kan i vissa fall få annat betydelse, exempelvis att en extern font behöver laddas ned först i stället för att länkas till.

CSP skyddar mot säkerhetshot som Cross-Site Scripting (där angripare försöker initierar skadlig kod), data-stöld (genom att begränsa vilka externa tjänster kan nås), malware-distribution (genom att hindra från att farlig kod laddas in).

När jag bytte från innerHTML till textContent visades användarens inmatning fortfarande på sidan som vanlig text, men utan att tolkas som HTML. Det minskade risken för DOM-baserad XSS. När jag därefter lade till en Content Security Policy slutade knappen först att fungera, eftersom min kod använde inline-JavaScript i både onclick och ett <script>-block. CSP blockerade alltså denna typ av skriptkörning. För att få både funktion och säkerhet behöver JavaScript därför flyttas till en extern fil och händelsen kopplas med addEventListener. Detta ger ett bättre skydd i flera lager.

Säkerhetsåtgärd 3

Som ytterligare skydd kan inmatningen valideras eller filtreras innan den visas, exempelvis genom att ersätta tecken som “<“ och “>” i JS-koden så att de inte kan tolkas som kod. Användarens input visas redan som text, så sanitize-funktionen är mer ett extra skyddslager.

Säkerheten bygger alltså på flera lager: säker DOM-hantering, begränsning av resurser via CSP och försiktig hantering av användarinmatning.

Tillgänglighet

Vi byggande av webbsidan behöver de fyra WCAG principerna finnas. All information och användargränssnitt på sidan behöver kunna uppfattas (“perceivable”), användare måste kunna hantera alla användargränssnitt och navigering (“operable”), och kunna förstå alla information och alla användargränssnitt på sidan (“understandable”), till exempel om det inträffar ett fel när användaren klickar på “Submit” button och användaren får ett felmeddelande, behöver hen kunna förstå vad felet betyder och vad man ska göra för att komma vidare med det.[3]

Navigering via <tab>
Man knappar oftast mellan länkar, knappar, formulärfält och andra interaktiva element. På den här sidan går det att tabba mellan URL-fältet, inputfältet och knappen. Sidan bör använda semantiskt HTML, såsom <button>, <section> med flera, och jag har lagt in flera <html> element som förslag, vilka är att

[3] Belfrage Paul, Webbutveckling 1, 2025, s. 196

föredra framför icke-semanstisk HTML element t.ex. .

`Fokus -`

Den fjärde WCAG principen, robusthet, ställer krav på att sidans innehåll ska vara tillgänglig med hjälpmedel och andra användaragenter och att arbete med tillgänglighet behöver vara dynamisk och webbsidans teknik hänga med när dessa tekniker utvecklas.

I dagsläget, år 2026 använder användare sig av exempelvis skärmläsare NVDA (vid Windows).[4] Personer som använder skärmsläkare kan ofta hoppa mellan landmarks, regioner, rubriker. När en sektion är märkt med aria-label så blir navigationen tydligare. Knappen av mig märkt med id="button-m" markerade jag först med `aria-label` för skärmläsare, vartefter jag tog bort markeringen eftersom jag bedömde att knappen redan har ett namn genom sin synliga text "Skicka", vilket är tillräckligt uttrycksfullt för en skärmläsare.

Tabindex="0" bör användas när ett ickelement behöver förtydligas ytterligare. Att lägga in på sidan och tabindex ="0" hade dock varit överflödigt sett mot sidans nuvarande innehåll.

Bilden på sidan har fått en alt-beskrivning.

Färgkontrast:

- - - text: #111111
  - - card-bg: #f5f5f5
  Contrast-ratio: xxxx

[4] NVDA, https://www.nvaccess.org/

(Testat på webaim.com)[5]

- - - text: #11111
  - - button-bg: rgb(250, 242, 211) /

  vilket uppfyller WCAG princip om att informationen ska vara upptäckbar.

  Radavstånd mellan linjer bör vara minst 1.5 (satt i <body> i CSS).

Hållbarhet

Energikrävande animation har tagits bort. Den anses dessutom inte fylla någon annan en eventuell en estetisk funktion, men tar för mycket av uppmärksamhet från innehållet i sidan och är inte ändamålsenligt. Vill man ha animationen kvar bör bakgrunder följa organisationens grafiska profil. Om bakgrundskortet (.card) tas bort eller dess bakgrund ändras behöver nya färgkontrast tester göras mellan text och bakgrund för att uppfylla krav på tillgänglighet.

Bilder står ofta för den största datamängden på en webbsida. Jag laddade ned en bild med skrivmaskin på ett skrivbord i mediumformat (800 x 450) på 81KB och omvandlade denna via sqoosh.app till WebP för att minska filstorleken. Med Quality minskad till 75% blev och nytt filformat blev besparingen på 59% till 33KB.

Jag använde även attributet `loading="lazy"` så att bilden bara laddas när den behövs.

`loading="lazy"` i html innebär att bilden laddas först efter det att html har

[5] Contrast Checker, https://webaim.org/resources/contrastchecker/

laddats ned, vilket är bra för sidans prestanda, det vill säga nedladdningstid **(men sparar inte på energi?)**

Kodoptimering

En annan åtgärd för att minska energiförbrukningen är att minimera mängden data som skickas över nätverket. Detta kan göras genom minifiering av HTML, CSS och JavaScript. Vid minifiering tas onödiga mellanslag, radbrytningar och kommentarer bort från koden, vilket gör filerna mindre. Mindre filer innebär att mindre data behöver överföras mellan server och användare, vilket minskar både laddningstid och energiförbrukning.

(*I den här uppgiften minimiserade jag dock inte koden så att den skulle synas bättre för läraren.)*

Ett annat förslag för att minska miljöpåverkan är att sidan läggs på en server som drivs av göra energi (grön hosting).

Överensstämmelse med annan regelverk/lagstifning

Val av bilder

En personuppgift är allt som kan identifiera en specifik person inklusive kontext enligt General Data Protection Regulation (GDPR). Som exempel valde jag en bild utan personer, eftersom fotografier av identifierbara människor kan räknas som personuppgifter. Genom att använda en bild på ett objekt undviks potentiella integritetsproblem. Bilden bör helst passa till den information man vill förmedla, och det kan också vara av symbolisk art. Ur tillgänglighetssynpunkt behöver alla bilder ha en `alt-tag`.

Bilden på sidan och bakgrundbilden faller under lagen (1960:729) om rätt till litterära och

konstnärliga verk (URL). Kan bilderna klassas som konstverk och uppfyller så kallad verkshöjd blir de skyddade som verk. Fotografiet på skrivmaskinen, i den mån fotot är tagen med en kamera kan eventuellt bli skyddad som så kallad närstående rättighet enligt fotorätten i 49 §URL, om fotografiet inte anses vara ett verk. I bägge fall har skaparen en ideell och ekonomisk rätt och ensamrätt att förfoga över sina bilder, och vari ingår att mångfaldiga/kopiera bilder, bearbeta dessa, visa dessa och sprida dessa offentligt i olika medier. Skyddstiden för upphovsrätten gäller 70 år efter upphovsmannens död, och för närstående rättigheten i 50 år efter det att bilden togs.

När en webbläsare laddar ned en bild skapas en kopia av bilden på användarens enhet, och bilden görs samtidigt tillgänglig för webbplatsens besökare.
Licenser är upphovsmännens och fotografers frivilliga avtal om att i mer eller mindre utsträckning erbjuda andra att förfoga över deras bilder utan att betala för det. Det finns olika typer av licenser med såväl friheter som inskränkningar i förfogandet för bilder, filmer, ljud och andra alster. [6]
Man behöver alltid kontrollera om nedladdningsplatsen erbjuder giltiga licenser, och om dessa inte finns be skaparen om lov före användning.

Bilden på sidan och bakgrundsbilden faller under Creative Commons, CC0-licens och de får både kopieras och visas offentligt även i kommersiellt syfte.

Upphovsrätt och närstående rättigheter

Datainsamling via formulär

Ur ett etiskt perspektiv är det viktigt att skydda användarens integritet och inte samla in mer information än nödvändigt. Därför använder formuläret endast ett enkelt meddelandefält och bara de personuppgifter bör samlas som behövs för ändamålet, såsom en första kontakt.

---

[6] Se exempel på licenser på https://creativecommons.org/share-your-work/cclicenses/

Användning av webbanalys-verktyg, information och samtycke

Enligt ePrivacy-direktivet behöver webbplatsen informera om och inhämta samtycke redan innan ett trafik-analysprogram, t.ex. Google Analytics eller en europeisk motsvarighet såsom Matomo, Plausible Analytics eller Fathom Analytics används. Besökaren behöver aktivt godkänna cookies som inte är att anse nödvändiga, med andra ord ska besökaren kunna neka statistikcookies.
Google Analytics är inte förbjudet enligt GDPR, eftersom programmet nu har automatisk IP-anonymisering, men som med vilket annat webbanalys-verktyg, man måste informera användaren vilka data som samlas in, varför, vem som får tillgång till dem, hur länge de sparas, och hur man kan säga nej.

Information och samtycke

Information till besökaren kan till exempel ske genom en cookie-banner och en integritetspolicy. Där anges vilka uppgifter som samlas in, exempelvis IP-adress, webbläsare, besökta sidor och besökstid. Det räcker dock inte med att informera besökaren. Analysverktyg får inte aktiveras förrän användaren har gett sitt samtycke. Dessförinnan behöver skriptet som laddar Google Analytics från Google Tag Manager/gtag.js stoppas.

Google Analytics implementeras vanligtvis med JavaScript-biblioteket gtag.js, vilket laddas från Googles servrar. Installationskoden tillhandahålls i Googles officiella utvecklardokumentation.[7] Där uppges dock den enklase JavaScript kod versionen som inte tar hänsyn till cookie-samtycke. Skriptet i <head> hade då aktiverats direkt när sidan laddas.

Se förslag på “Alternativ skript” i `script.js`. Först när användaren klickar på Acceptera så laddas t.ex. Googles bibliotek och gtag(‘config’) laddar analysprogrammet.

---

[7] Länk till installationssida från Google Analytics. Set up the Google tag with gtag.js. https://developers.google.com/tag-platform/gtagjs

Besökaren behöver även aktivt godkänna cookies för e-post insamling och marknadsföring. (??? Stämmer det?)

Även i övrigt måste GDPR följas, dvs. det ska finnas en rättslig grund för datainsamling, samtycke till att samla personuppgifter ska vara frivillig och aktiv, och det ska finnas enkla och snabba rutiner för att radera data så snart besökaren begär det.

Checkboxen vid "*Jag godkänner att mina uppgifter samlas för marknadsföring från Organisation AB*" är frivillig enligt GDPR. Användaren kan använda tjänsten utan att ge sitt samtycke till marknadsföring. Denna checkbox bör vara avmarkerad. Required-attributet bör inte heller användas här, eftersom samtycket är frivillig. I annat fall tvingas användaren att ge sitt samtycke till marknadsföring för att kunna skicka ett meddelande via formuläret. Detsamma hade gällt om användaren skulle samtycka till att använda en tjänst.

Insamling av uppgifter för att svara på meddelandet kan ses som en förutsättning för att kunna svara, och det markeras inte som obligatoriskt med required eller stjärna på samma sätt som ett frivilligt samtycke. Denna information bör i stället presenteras som ett informativt meddelande, t.ex. "Vi kommer att använda dina uppgifter för att svara på ditt meddelande", snarare än som ett val som användaren måste aktivt samtycka till.

Sända data till tredje part + samtycke?

Kortfattad reflektion om säkerhet, etik och hållbarhet.

I arbetet med webbplatsen försökte jag balansera säkerhet, tillgänglighet och hållbarhet.

Ur ett etiskt perspektiv är det viktigt att skydda användarens integritet och inte samla in mer information än nödvändigt. Exempel på insamlade data på den här sidan är: namn, epost och meddelande. Därför använder formuläret endast ett enkelt meddelandefält och lagrar ingen data. Jag valde även en bild utan personer för att undvika att personuppgifter eller identifierbara individer förekommer.

Hänsyn till hållbarhetsaspekten tas genom att optimera bilder och ta bort onödig animation som skulle kunna öka energiförbrukningen.

Jag valde en bild utan personer eftersom fotografier av identifierbara människor kan räknas som personuppgifter. Genom att använda en bild på ett objekt undviks potentiella integritetsproblem.